

Software Requirements Specification

PlayQuiz — A Console Java Quiz Application

Version 1.0 | 2026-10-01

Author: Dipsana

Format: IEEE Std 830-1998

Repository: java-mini-projects (branch j-play-quiz)

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions	3
1.4 References	3
1.5 Overview	3
2. Overall Description.....	3
2.1 Product Perspective	3
2.2 Product Functions (summary).....	4
2.3 User Characteristics	4
2.4 Constraints.....	4
2.5 Assumptions and Dependencies	4
3. Specific Requirements.....	4
3.1 Functional Requirements	4
3.2 External Interface Requirements	6
3.3 Performance Requirements	6
3.4 Design Constraints.....	6
3.5 Software System Attributes	6
4. Appendices	6
Appendix A: Repository Structure	6
Appendix B: Known Limitations and Observations.....	7
Appendix C: Release Information.....	7

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of PlayQuiz v1.0.0. It is intended for developers, contributors, and reviewers of the `java-mini-projects` repository.

1.2 Scope

PlayQuiz is a terminal-based, single-user multiple-choice quiz on Java fundamentals. It asks 78 questions, accepts answers in many informal formats, scores the session, and offers replay. It is distributed as native packages and a JAR.

Out of scope (assumed, to be confirmed): score persistence, GUI, multi-user or networked play, question randomization, timers, user accounts, external question files.

1.3 Definitions

Term	Meaning
Resolution	Mapping raw user input to option A–D, or to "skip"
Skip	Empty input; the question is unanswered and the correct answer is shown
EOF	End of input stream (e.g. Ctrl+D / Ctrl+Z)
jpackage	JDK tool for creating native installers/app-images

1.4 References

PlayQuiz `README.md`; `CHANGELOG.md` (v1.0.0); repository `README.md` and `STRUCTURE.md`; `docs/test-log.txt`; IEEE Std 830-1998.

1.5 Overview

Section 2 describes the product context, Section 3 the specific requirements, and Section 4 the appendices.

2. Overall Description

2.1 Product Perspective

A standalone console application. Component flow:



Package `com.dipsana.quiz`: `Main`, `model.Question`, `service.QuestionService`, `ui.QuizApp`.
Global helpers: `safeInput()` and `clearInput()`. The project lives on branch `j-play-quiz` of a branch-per-project repository (see Appendix A).

2.2 Product Functions (summary)

Welcome animation, 78-question quiz, flexible answer resolution, skip, scoring with tiered feedback, replay loop, graceful shutdown, cross-platform build and packaging.

2.3 User Characteristics

Java learners and hobbyists comfortable with a terminal. No technical setup is needed when using packaged builds.

2.4 Constraints

- Java 21 LTS, Maven 3.9+ (source builds); PowerShell 7+ for the build dispatcher.
- Windows builds need an elevated shell (Chocolatey).
- Unsigned binaries trigger Windows SmartScreen and macOS Gatekeeper warnings.
- macOS/Linux builds are untested by the author.
- Input is capped at 1000 characters.
- The test source folder is currently empty (no automated tests).

2.5 Assumptions and Dependencies

- Packaged builds bundle a JVM; the JAR needs Java 21+.
- GitHub Actions builds all three platforms on tagged releases.
- Bootstrap depends on Chocolatey (Windows) and SDKMAN (macOS/Linux).

3. Specific Requirements

3.1 Functional Requirements

Welcome screen

FR-1 The system shall display an animated welcome sequence, printing one word at a time.

FR-2 Before each word, the system shall pause 500 ms and discard pending typed-ahead input, so users cannot skip the intro accidentally.

FR-3 The welcome shall state the question count (78), that Enter skips, and the 1000-character limit.

Question presentation

FR-4 The system shall present each question with its number, stem, four options (A–D), and the prompt "Do you know the answer (a/b/c/d)?".

FR-5 Questions shall be presented in fixed order, Q1 to Q78. (Assumed: no randomization.)

FR-6 Stems may contain multi-line code snippets, shown verbatim.

Answer resolution

FR-7 The system shall accept an answer by option letter (case-insensitive), by option number 1–4, by ordinal word or abbreviation (first, 1st, one, fourth, 4th), and by opt/option prefixes (e.g., opt 3, option c).

- FR-8** The system shall ignore whitespace within the input (o p t i o n c → C).
- FR-9** The system shall accept repeated characters (BBBBBBBBBB → B; ddddddddddDD DDDD → D).
- FR-10** The system shall accept the full text of an answer, ignoring letter case and surrounding whitespace, but shall not accept answer text containing extra or missing characters.
- FR-11** After resolving, the system shall echo You entered: <option text>.
- FR-12** If the input cannot be resolved to exactly one option, or refers to multiple different options, the question shall be treated as incorrect and the correct answer shown.
- FR-13** Leading or trailing symbols and noise shall be tolerated where a valid token is present (! 2 `` _ - -- → B).

Skip

- FR-14** Empty input shall skip the question, show "Don't worry! I got you @(> ^ <)@" and the correct answer, and count as unanswered (0 points).

Feedback and scoring

- FR-15** A correct answer shall display "That's correct!" and add 1 point.
- FR-16** A wrong answer shall display "Oh-ooh! You got it wrong >w' _ 'w<" and the correct answer.
- FR-17** At the end, the system shall show Final Score: X out of 78 and the percentage to two decimals.
- FR-18** The system shall display a feedback message based on the final percentage:
- 90% and above: "Brilliant! You aced it!"
 - 50% to below 90%: "Nice!"
 - 25% to below 50%: "Well tried!"
 - Below 25%: "You need to study harder!"

Replay

- FR-19** After results, the system shall ask "Do you want to play again (y/n)?".
- FR-20** On invalid replay input (e.g., hello hi, why), the system shall re-prompt.
- FR-21** Shall restart the quiz with a fresh score; n shall display the goodbye message ("I hope you had fun... Have a wonderful day @^^@"), wait 6 seconds and exit.
- FR-22** Replay input is matched leniently: nooo exited, so a leading y/n is accepted.

Robustness

- FR-23** On Ctrl+C or EOF, the system shall print "Calm down! Shutting down gracefully..." and exit without a stack trace.
- FR-24** safeInput() shall handle newlines, EOF, and over-long input (1000-char cap) without failing.
- FR-25** clearInput() shall flush pending input between reads.

Build and distribution

- FR-26** build.ps1 shall detect the OS and run the correct bootstrap/build script, self-elevating on Windows via UAC.

FR-27 Bootstrap scripts shall install Java 21 and Maven if missing (Chocolatey on Windows, SDKMAN on macOS/Linux) and shall be idempotent.

FR-28 The build shall produce a JAR and a native package via jpackage: Windows app-image .exe, macOS .dmg, Linux .deb, into installer/.

FR-29 Native packages shall bundle a JVM so no Java install is needed.

FR-30 A GitHub Actions workflow (build-playquiz.yml) shall build all three platforms on each tagged release.

3.2 External Interface Requirements

- **User interface:** console text only. Emoticons used:
`@' '@, @(> ^ <)@, @^^@, > ' ' <, >w' _ 'w<.`
- **Hardware:** none beyond a terminal-capable machine.
- **Software:** JDK 21+, Maven 3.9+, PowerShell 7+, jpackage, Chocolatey or SDKMAN, Git/GitHub.
- **Communications:** none at runtime (network is used only during bootstrap and release).

3.3 Performance Requirements

PR-1 Input resolution shall feel instantaneous (no perceptible delay).

PR-2 The welcome animation delay is fixed at 500 ms per word.

PR-3 Memory use is minimal; input is bounded at 1000 characters.

3.4 Design Constraints

- Layered structure: UI / service / model.
- Standard Java I/O (BufferedReader over System.in); no third-party runtime dependencies.
- Repository rules in Appendix A apply.

3.5 Software System Attributes

- **Usability:** forgiving input; friendly, non-punishing tone.
- **Reliability:** no crash on Ctrl + C, EOF, empty, garbage input, and resilient to intentional Out of Memory Inputs.
- **Portability:** Windows (tested), macOS and Linux (untested).
- **Maintainability:** small codebase; MVC-like separation; MIT license.
- **Security:** no network use, no stored data. Unsigned binaries are expected to show OS warnings.

4. Appendices

Appendix A: Repository Structure

Branch-per-project: main (aggregate, shared root files), new (locked, always empty template), and j-
<kebab-case> branches each holding a single <PascalCase>/ folder. Rules: no root files on project

branches, never edit main directly (PRs only), never edit the template, and delete and recreate polluted branches. Each project carries its own LICENSE.

Appendix B: Known Limitations and Observations

1. **Answer Resolution Design Notes.** The answer parser is intentionally strict in some places and forgiving in others.
 - Repeated references to the same option are recognized (e.g., fourth four four option opt opt D → D). This is handled by regex normalization.
 - Digits outside the range 1–4 are ignored, and the first digit in range selects the option (200 → B, 895781 → A).
 - Answer-text matching is exact apart from letter case and surrounding whitespace. Extra or missing characters are rejected, because many questions are output-based and a near-match would not be a valid answer (e.g., int int is not accepted for int).
 - Input that refers to several different options (e.g., a b c d) is not resolved and counts as incorrect, because the intent is unclear.
 - Input with no recognizable option reference (99, gugu gaga, answer is c) is likewise treated as incorrect.
2. **Docs.** docs/SRS.pdf and SRS.docx are listed in the folder tree; this document is meant to supply their content.
3. **No automated tests** exist yet (src/test/java is empty).

Appendix C: Release Information

v1.0.0, 2026-10-01. License: MIT.